SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

Smohit Arman Swain
192572092RC

CO4_AT2_CASE STUDY

← Back

QUESTIONS

Q1 ✓
Student Performance Analysis — List + Tuple
10 marks

Q2 ✓
Customer Name Analysis — String + List
10 marks

Q3 ✓
Product Inventory Analysis — List + Tuple
10 marks

Q4 ✓
Text Analysis — String + List
10 marks

Q5 ✓
Employee Skill Analysis — Tuple + List + String
10 marks

Q6 ✓
Transaction Analysis — Tuple + List
10 marks

Q7 ✓
Password Security Analysis — String + List
10 marks

Q10 Integrated Student Data Analysis — String + List + Tuple

10 marks

A college maintains:
students = [
(" A001 ", "ARUN KUMAR", [78, 85, 92]),
("A002", "Bala", [45, 67, 52]),
(" a003", "arun kumar", [88, 91, 95]),
("A004", "CHARAN", [72, 76, 81]),
("A005", "Divya ", [55, 48, 62])
]
Each record contains:
(Student ID, Student Name, Marks)
Analyze the complete dataset:
1. Normalize all student IDs using strip() and upper().
2. Normalize all student names using strip() and lower().
3. Calculate total and average marks.
4. Identify students who passed all subjects.
5. Identify students who failed at least one subject.
6. Find the highest total.
7. Find the lowest total.
8. Identify students with average above 75.
9. Detect duplicate student names after normalization.
10. Sort students based on average marks.
11. Find the top two students.
12. Find the subject-wise highest marks.

Q8
Code Analysis and Debugging — List + Tuple + String
10 marks

Q9
Word Frequency and Data Analysis — String + List + Tuple
10 marks

Q10
Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

13. Find the subject-wise average marks.
14. Count the number of students who scored above 80 in at least two subjects.
15. Identify the student who has the highest mark in Subject 3.
16. Explain which data structure is appropriate for student records, marks, and individual student identity, and justify your answer.

Expected Output:

Normalized Records:

A001 arun kumar [78, 85, 92]
A002 bala [45, 67, 52]
A003 arun kumar [88, 91, 95]
A004 charan [72, 76, 81]
A005 divya [55, 48, 62]

Totals:
arun kumar → 255
bala → 164
arun kumar → 274
charan → 229
divya → 165

Averages:
arun kumar → 85.00
bala → 54.67
arun kumar → 91.33
charan → 76.33
divya → 55.00

Passed All Subjects:
arun kumar
arun kumar
charan

Failed At Least One:
bala
divya

Highest Total:
arun kumar → 274

Lowest Total:

bala → 164

Average > 75:

arun kumar

arun kumar

charan

Duplicate Name:

arun kumar

Top 2:

arun kumar → 91.33

arun kumar → 85.00

Subject-wise Highest:

Subject 1 → 88

Subject 2 → 91

Subject 3 → 95

Subject-wise Average:

Subject 1 → 67.60

Subject 2 → 73.40

Subject 3 → 78.40

Your Code

```
students = [
    (" A001 ", "ARUN KUMAR", [78, 85, 92]
    ("A002", "Bala", [45, 67, 52]),
    (" a003", "arun kumar", [88, 91, 95])
    ("A004", "CHARAN", [72, 76, 81]),
    ("A005", "Divya ", [55, 48, 62])
]

normalized = []

# 1, 2 and 3. Normalize ID, name, total a
for record in students:
    student_id = record[0].strip().upper(
    name = record[1].strip().lower()
    marks = record[2]

    total = marks[0] + marks[1] + marks[2
    average = total / 3

```

```
    normalized.append((student_id, name,

print("Normalized Records:")
for record in normalized:
    print(record[0], record[1], record[2]

# Totals
print("\nTotals:")
for record in normalized:
    print(record[1], "->", record[3])

# Averages
print("\nAverages:")
for record in normalized:
    print(record[1], "->", "%.2f" % recor


# 4. Students who passed all subjects
print("\nPassed All Subjects:")
for record in normalized:
    marks = record[2]

    if marks[0] >= 40 and marks[1] >= 40
        print(record[1])


# 5. Students who failed at least one sub
print("\nFailed At Least One:")
for record in normalized:
    marks = record[2]

    if marks[0] < 40 or marks[1] < 40 or
        print(record[1])


# 6. Highest total
highest = normalized[0]

for record in normalized:
    if record[3] > highest[3]:
        highest = record

print("\nHighest Total:")
print(highest[1], "->", highest[3])


# 7. Lowest total
lowest = normalized[0]
```

```
68
69 for record in normalized:
70     if record[3] < lowest[3]:
71         lowest = record
72
73 print("Lowest Total:")
74 print(lowest[1], "->", lowest[3])
75
76
77 # 8. Average above 75
78 print("\nAverage > 75:")
79 for record in normalized:
80     if record[4] > 75:
81         print(record[1])
82
83
84 # 9. Duplicate student names
85 print("\nDuplicate Name:")
86
87 names = []
88
89 for record in normalized:
90     if record[1] not in names:
91         names.append(record[1])
92     else:
93         print(record[1])
94
95
96 # 10. Sort students based on average
97 sorted_students = normalized[:]
98
99 for i in range(len(sorted_students)):
100     for j in range(i + 1, len(sorted_stud
101         if sorted_students[i][4] > sorted
102             temp = sorted_students[i]
103             sorted_students[i] = sorted_s
104             sorted_students[j] = temp
105
106 print("\nStudents Sorted by Average:")
107 for record in sorted_students:
108     print(record[1], "->", "%.2f" % recor
109
110
111 # 11. Top two students
112 print("\nTop 2:")
113
114 count = 0
115
```

```
for i in range(len(sorted_students) - 1,
    print(sorted_students[i][1], "->", "%
    count = count + 1

    if count == 2:
        break


# 12. Subject-wise highest marks
subject1 = normalized[0][2][0]
subject2 = normalized[0][2][1]
subject3 = normalized[0][2][2]

for record in normalized:
    marks = record[2]

    if marks[0] > subject1:
        subject1 = marks[0]

    if marks[1] > subject2:
        subject2 = marks[1]

    if marks[2] > subject3:
        subject3 = marks[2]

print("\nSubject-wise Highest:")
print("Subject 1 ->", subject1)
print("Subject 2 ->", subject2)
print("Subject 3 ->", subject3)


# 13. Subject-wise average
sum1 = 0
sum2 = 0
sum3 = 0

for record in normalized:
    marks = record[2]

    sum1 = sum1 + marks[0]
    sum2 = sum2 + marks[1]
    sum3 = sum3 + marks[2]

print("\nSubject-wise Average:")
print("Subject 1 ->", "%.2f" % (sum1 / 5))
print("Subject 2 ->", "%.2f" % (sum2 / 5))
print("Subject 3 ->", "%.2f" % (sum3 / 5))

```

```
164
165 # 14. Students scoring above 80 in at lea
166 count = 0
167
168 for record in normalized:
169     marks = record[2]
170     above80 = 0
171
172     for mark in marks:
173         if mark > 80:
174             above80 = above80 + 1
175
176     if above80 >= 2:
177         count = count + 1
178
179 print("\nStudents scoring above 80 in at
180 print(count)
181
182
183 # 15. Student with highest Subject 3 mark
184 highest_subject3 = normalized[0]
185
186 for record in normalized:
187     if record[2][2] > highest_subject3[2]
188         highest_subject3 = record
189
190 print("\nHighest Subject 3:")
191 print(highest_subject3[1], "->", highest_
192
193
194 # 16. Data structure explanation
195 print("\nData Structure:")
196 print("Tuple is suitable for each student
197 print("name and marks belong together.")
198 print("List is suitable for storing all s
199 print("Dictionary can be used when quick
```

Input

stdin input

Output

Run Save

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

Smohit Arman Swain
192572092RC

CO4_AT2_CASE STUDY

← Back

QUESTIONS

Q1 ✓
Student Performance Analysis — List + Tuple
10 marks

Q2 ✓
Customer Name Analysis — String + List
10 marks

Q3 ✓
Product Inventory Analysis — List + Tuple
10 marks

Q4 ✓
Text Analysis — String + List
10 marks

Q5 ✓
Employee Skill Analysis — Tuple + List + String
10 marks

Q6 ✓
Transaction Analysis — Tuple + List
10 marks

Q7 ✓
Password Security Analysis — String + List
10 marks

Q9 Word Frequency and Data Analysis — String + List + Tuple

10 marks

Consider:

text = "data science uses data analysis and data visualization"

Analyze the string using Python.

Tasks

1. Convert the sentence into a list of words.
2. Count the total words.
3. Find unique words.
4. Find repeated words.
5. Count the frequency of every word.
6. Identify the most frequently occurring word.
7. Create tuples of the form:

(word, frequency)

8. Sort the tuples based on frequency.
9. Find words having frequency greater than 1.
10. Identify the longest word.
11. Find words beginning with 'd'.
12. Calculate the percentage of repeated words.

Expected Output:

Total Words: 9

Unique Words:

['data', 'science', 'uses', 'analysis', 'and', 'visualization']

Frequency:

data → 3

Q8
Code Analysis and Debugging — List + Tuple + String
10 marks

Q9
Word Frequency and Data Analysis — String + List + Tuple
10 marks

Q10
Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

science → 1

uses → 1

analysis → 1

and → 1

visualization → 1

Most Frequent:

data

Repeated Words:

data

Longest Word:

visualization

Your Code

```
text = "data science uses data analysis a

# 1. List of words
words = text.split()
print("Words:", words)

# 2. Total words
print("Total Words:", len(words))

# 3 and 5. Unique words and frequency
unique = []
frequency = []

for word in words:
    if word not in unique:
        unique.append(word)
        frequency.append(1)
    else:
        pos = unique.index(word)
        frequency[pos] = frequency[pos] +

print("Unique Words:")
print(unique)

# 5. Frequency
print("Frequency:")
for i in range(len(unique)):
    print(unique[i], "->", frequency[i])

# 4. Repeated words
```

```
print("Repeated Words:")
for i in range(len(unique)):
    if frequency[i] > 1:
        print(unique[i])

# 6. Most frequent word
highest = frequency[0]
most = unique[0]

for i in range(len(frequency)):
    if frequency[i] > highest:
        highest = frequency[i]
        most = unique[i]

print("Most Frequent:")
print(most)

# 7. Tuples
tuples = []

for i in range(len(unique)):
    tuples.append((unique[i], frequency[i

print("Tuples:")
print(tuples)

# 8. Sort tuples by frequency
for i in range(len(tuples)):
    for j in range(i + 1, len(tuples)):
        if tuples[i][1] > tuples[j][1]:
            temp = tuples[i]
            tuples[i] = tuples[j]
            tuples[j] = temp

print("Sorted Tuples:")
print(tuples)

# 9. Frequency greater than 1
print("Frequency greater than 1:")
for i in range(len(unique)):
    if frequency[i] > 1:
        print(unique[i])

# 10. Longest word
longest = words[0]

for word in words:
    if len(word) > len(longest):
```

```
        longest = word

print("Longest Word:")
print(longest)

# 11. Words beginning with d
print("Words beginning with d:")
for word in unique:
    if word[0] == 'd':
        print(word)

# 12. Percentage of repeated words
repeated = 0

for i in range(len(unique)):
    if frequency[i] > 1:
        repeated = repeated + 1

percentage = (repeated / len(unique)) * 1

print("Percentage of Repeated Words:")
print("%.2f" % percentage, "%")
```

Input

stdin input

Output

Run Save

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

Smohit Arman Swain
192572092RC

CO4_AT2_CASE STUDY

← Back

QUESTIONS

Q1 ✓
Student Performance Analysis — List + Tuple
10 marks

Q2 ✓
Customer Name Analysis — String + List
10 marks

Q3 ✓
Product Inventory Analysis — List + Tuple
10 marks

Q4 ✓
Text Analysis — String + List
10 marks

Q5 ✓
Employee Skill Analysis — Tuple + List + String
10 marks

Q6 ✓
Transaction Analysis — Tuple + List
10 marks

Q7 ✓
Password Security Analysis — String + List
10 marks

Q8 **Code Analysis and Debugging — List + Tuple + String** 10 marks

Consider the following program:

```
records = [
(" Arun ", "Python"),
("ARUN", "Python"),
(" Bala ", "Java"),
("bala", "Java"),
("Charan", "Python")
]

names = []

for record in records:
name = record[0].strip().lower()

if name not in names:
names.append(name)

print(names)
```

Analyze the program:

1. What happens during each iteration?
2. What value is stored in record?
3. What does record[0] represent?
4. Why are strip() and lower() used together?
5. Why does "Arun" appear only once?
6. Why does "Bala" appear only once?
7. What is the final output?
8. Modify the logic to find unique employees and

Q8
Code Analysis and Debugging — List + Tuple + String
10 marks

Q9
Word Frequency and Data Analysis — String + List + Tuple
10 marks

Q10
Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

their skills.

9. Analyze what happens if record[0] is changed instead.

10. Explain why modifying record[0] directly is not allowed.

Expected Output:

['arun', 'bala', 'charan']

Your Code

```
records = [
    (" Arun ", "Python"),
    ("ARUN", "Python"),
    (" Bala ", "Java"),
    ("bala", "Java"),
    ("Charan", "Python")
]

# 1. Find unique employee names
names = []

for record in records:
    name = record[0].strip().lower()

    print("Record:", record)
    print("Name:", name)

    if name not in names:
        names.append(name)

print("\nUnique Employee Names:")
print(names)


# 8. Find unique employees and their skil
employees = {}

for record in records:
    name = record[0].strip().lower()
    skill = record[1]

    if name not in employees:
        employees[name] = skill

print("\nUnique Employees and Skills:")
for name in employees:
```

```
    print(name, "->", employees[name])


# 9 & 10. Tuples cannot be modified direct
print("\nOriginal Record:")
record = ("Arun", "Python")
print(record)

# record[0] = "arun"   # This gives TypeEr
```

Input

stdin input

Output

Run Save

CO4_AT2_CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis

Smohit Arman Swain
192572092RC

10 marks

Q4
Text Analysis — String + List
10 marks

Q5
Employee Skill Analysis — Tuple + List + String
10 marks

Q6
Transaction Analysis — Tuple + List
10 marks

Q7
Password Security Analysis — String + List
10 marks

Q7 **Password Security Analysis — String + List** 10 marks

Admin@2026
]
A valid password must have:
• At least 8 characters
• Uppercase
• Lowercase
• Digit
• Special character
Analyze:
1. Check every password against all five conditions.
2. Identify valid passwords.
3. Identify invalid passwords.
4. For each invalid password, identify the failed conditions.
5. Count valid and invalid passwords.
6. Find the longest password.
7. Find passwords containing "python" irrespective of

Q8
Code Analysis and Debugging — List + Tuple + String
10 marks

Q9
Word Frequency and Data Analysis — String + List + Tuple
10 marks

Q10
Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

case.

8. Sort passwords based on length.

9. Determine the percentage of valid passwords.

Expected Output:

Valid:

Python@123

Java#4567

Admin@2026

Invalid:

python123 → No uppercase, No special character

PYTHON@123 → No lowercase

Py@45 → Less than 8 characters

Python123 → No special character

Valid Count: 3

Invalid Count: 4

Longest Password:

Python@123

Python-related passwords:

Python@123

python123

PYTHON@123

Python123

Valid Percentage: 42.86%

Your Code

```
passwords = [
    "Python@123",
    "python123",
    "PYTHON@123",
    "Py@45",
    "Python123",
    "Java#4567",
    "Admin@2026"
]

valid = []
invalid = []

for p in passwords:
    upper = False
```

```
    lower = False
    digit = False
    special = False

    for ch in p:
        if ch >= 'A' and ch <= 'Z':
            upper = True
        elif ch >= 'a' and ch <= 'z':
            lower = True
        elif ch >= '0' and ch <= '9':
            digit = True
        else:
            special = True

    errors = []

    if len(p) < 8:
        errors.append("Less than 8 charact
    if upper == False:
        errors.append("No uppercase")
    if lower == False:
        errors.append("No lowercase")
    if digit == False:
        errors.append("No digit")
    if special == False:
        errors.append("No special characte

    if len(errors) == 0:
        valid.append(p)
    else:
        invalid.append([p, errors])

print("Valid:")
for p in valid:
    print(p)

print("Invalid:")
for item in invalid:
    print(item[0], "->", ", ".join(item[1

print("Valid Count:", len(valid))
print("Invalid Count:", len(invalid))

longest = passwords[0]

for p in passwords:
    if len(p) > len(longest):
        longest = p
```

```
64
65 print("Longest Password:")
66 print(longest)
67
68 print("Python-related passwords:")
69 for p in passwords:
70     if "python" in p.lower():
71         print(p)
72
73 print("Passwords sorted by length:")
74 sorted_passwords = sorted(passwords, key=l
75
76 for p in sorted_passwords:
77     print(p)
78
79 percentage = (len(valid) * 100) / len(pass
80
81 print("Valid Percentage: {:.2f}%".format(p
```

Input

stdin input

Output

Run Save

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

Smohit Arman Swain
192572092RC

QUESTIONS

Q1
Student Performance Analysis — List + Tuple
10 marks

Q2
Customer Name Analysis — String + List
10 marks

Q3
Product Inventory Analysis — List + Tuple
10 marks

Q4
Text Analysis — String + List
10 marks

Q5
Employee Skill Analysis — Tuple + List + String
10 marks

Q6
Transaction Analysis — Tuple + List
10 marks

Q7
Password Security Analysis — String + List
10 marks

Q6 Transaction Analysis — Tuple + List — 10 marks

A bank stores transactions:
transactions = [
("T101", "Deposit", 5000),
("T102", "Withdraw", 2000),
("T103", "Deposit", 8000),
("T104", "Withdraw", 1500),
("T105", "Deposit", 3000),
("T106", "Withdraw", 7000)
]
Analyze the transaction history.
Tasks
1. Calculate total deposits.
2. Calculate total withdrawals.
3. Calculate the net balance.
4. Find the largest deposit.
5. Find the largest withdrawal.
6. Identify all transactions greater than ₹4,000.
7. Count deposits and withdrawals.
8. Find the average transaction amount.
9. Sort transactions based on amount.
10. Identify whether total withdrawals exceed 50% of total deposits.
Expected Output:
Total Deposit: 16000
Total Withdrawal: 10500
Net Balance: 5500

Q8
Code Analysis and Debugging — List + Tuple + String
10 marks

Q9
Word Frequency and Data Analysis — String + List + Tuple
10 marks

Q10
Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

Largest Deposit: 8000

Largest Withdrawal: 7000

Transactions > 4000:

T101

T103

T106

Deposits: 3

Withdrawals: 3

Average Transaction: 4416.67

Withdrawals > 50% of Deposits:

Yes

Your Code

```
transactions = [
    ("T101", "Deposit", 5000),
    ("T102", "Withdraw", 2000),
    ("T103", "Deposit", 8000),
    ("T104", "Withdraw", 1500),
    ("T105", "Deposit", 3000),
    ("T106", "Withdraw", 7000)
]

# 1. Calculate total deposits
total_deposit = 0

for tid, transaction_type, amount in tran
    if transaction_type == "Deposit":
        total_deposit += amount

print("Total Deposit:", total_deposit)

# 2. Calculate total withdrawals
total_withdrawal = 0

for tid, transaction_type, amount in tran
    if transaction_type == "Withdraw":
        total_withdrawal += amount

print("Total Withdrawal:", total_withdraw

# 3. Calculate net balance
net_balance = total_deposit - total_withd
```

```
30
31 print("Net Balance:", net_balance)
32
33 # 4. Find largest deposit
34 deposits = []
35
36 for tid, transaction_type, amount in tran
37     if transaction_type == "Deposit":
38         deposits.append(amount)
39
40 largest_deposit = max(deposits)
41
42 print("Largest Deposit:", largest_deposit
43
44 # 5. Find largest withdrawal
45 withdrawals = []
46
47 for tid, transaction_type, amount in tran
48     if transaction_type == "Withdraw":
49         withdrawals.append(amount)
50
51 largest_withdrawal = max(withdrawals)
52
53 print("Largest Withdrawal:", largest_with
54
55 # 6. Transactions greater than 4000
56 print("\nTransactions > 4000:")
57
58 for tid, transaction_type, amount in tran
59     if amount > 4000:
60         print(tid)
61
62 # 7. Count deposits and withdrawals
63 deposit_count = 0
64 withdrawal_count = 0
65
66 for tid, transaction_type, amount in tran
67     if transaction_type == "Deposit":
68         deposit_count += 1
69     else:
70         withdrawal_count += 1
71
72 print("\nDeposits:", deposit_count)
73 print("Withdrawals:", withdrawal_count)
74
75 # 8. Average transaction amount
76 total_amount = 0
77
```

```
for tid, transaction_type, amount in tran
    total_amount += amount

average = total_amount / len(transactions

print("Average Transaction:", format(aver

# 9. Sort transactions based on amount
sorted_transactions = sorted(
    transactions,
    key=lambda transaction: transaction[2
)

print("\nTransactions Sorted by Amount:")

for transaction in sorted_transactions:
    print(transaction)

# 10. Check whether withdrawals exceed 50
if total_withdrawal > (total_deposit * 0.
    print("\nWithdrawals > 50% of Deposit
    print("Yes")
else:
    print("\nWithdrawals > 50% of Deposit
    print("No")
```

Input

stdin input

Output

Run Save

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

Smohit Arman Swain
192572092RC

QUESTIONS

Q5 Employee Skill Analysis — Tuple + List + String

10 marks

An organization stores:

employees = [

("E101", "Arun", ["Python", "SQL", "ML"]),

("E102", "Bala", ["Java", "SQL", "HTML"]),

("E103", "Charan", ["Python", "ML", "SQL"]),

("E104", "Divya", ["Python", "Java", "Cloud"]),

("E105", "Esha", ["Python", "ML", "Cloud"])

]

Analyze the employee skill data.

Tasks

1. Find all employees who know Python.
2. Find employees who know both Python and ML.
3. Find employees who know SQL but not Java.
4. Find employees who know either Java or Cloud.
5. Count employees having each skill.
6. Identify the most common skill.
7. Identify employees having at least three skills.
8. Find employees who do not know Python.
9. Sort employees based on the number of skills.
10. Determine which employees are suitable for a Python + ML project.

Expected Output:

Python:

Arun, Charan, Divya, Esha

Q8
Code Analysis and Debugging — List + Tuple + String
10 marks

Q9
Word Frequency and Data Analysis — String + List + Tuple
10 marks

Q10
Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

Python + ML:
Arun, Charan, Esha

SQL but not Java:
Arun, Charan

Java or Cloud:
Bala, Divya, Esha

Employees with 3 skills:
Arun, Bala, Charan, Divya, Esha

Without Python:
Bala

Most Common Skill:
Python

Suitable for Python + ML:
Arun, Charan, Esha

Your Code

```
employees = [
    ("E101", "Arun", ["Python", "SQL", "M
    ("E102", "Bala", ["Java", "SQL", "HTM
    ("E103", "Charan", ["Python", "ML", "
    ("E104", "Divya", ["Python", "Java",
    ("E105", "Esha", ["Python", "ML", "Cl
]

# 1. Employees who know Python
print("Python:")

for emp_id, name, skills in employees:
    if "Python" in skills:
        print(name, end=", ")

print()

# 2. Employees who know both Python and M
print("\nPython + ML:")

for emp_id, name, skills in employees:
    if "Python" in skills and "ML" in ski
        print(name, end=", ")

print()
```

```
26
27 # 3. Employees who know SQL but not Java
28 print("\nSQL but not Java:")
29
30 for emp_id, name, skills in employees:
31     if "SQL" in skills and "Java" not in
32         print(name, end=", ")
33
34 print()
35
36 # 4. Employees who know either Java or Cl
37 print("\nJava or Cloud:")
38
39 for emp_id, name, skills in employees:
40     if "Java" in skills or "Cloud" in ski
41         print(name, end=", ")
42
43 print()
44
45 # 5. Count employees having each skill
46 skill_count = {}
47
48 for emp_id, name, skills in employees:
49     for skill in skills:
50         if skill in skill_count:
51             skill_count[skill] += 1
52         else:
53             skill_count[skill] = 1
54
55 print("\nEmployees with each skill:")
56
57 for skill, count in skill_count.items():
58     print(skill, ":", count)
59
60 # 6. Most common skill
61 most_common_skill = max(skill_count, key=
62
63 print("\nMost Common Skill:")
64 print(most_common_skill)
65
66 # 7. Employees having at least three skil
67 print("\nEmployees with 3 skills:")
68
69 for emp_id, name, skills in employees:
70     if len(skills) >= 3:
71         print(name, end=", ")
72
73 print()
```

```
74
75 # 8. Employees who do not know Python
76 print("\nWithout Python:")
77
78 for emp_id, name, skills in employees:
79     if "Python" not in skills:
80         print(name, end=", ")
81
82 print()
83
84 # 9. Sort employees based on number of sk
85 sorted_employees = sorted(
86     employees,
87     key=lambda employee: len(employee[2])
88     reverse=True
89 )
90
91 print("\nEmployees sorted by number of sk
92
93 for emp_id, name, skills in sorted_employ
94     print(name, ":", len(skills))
95
96 # 10. Employees suitable for Python + ML
97 print("\nSuitable for Python + ML:")
98
99 for emp_id, name, skills in employees:
100     if "Python" in skills and "ML" in ski
101         print(name, end=", ")
102
103 print()
```

Input

stdin input

Output

Run Save

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

Smohit Arman Swain
192572092RC

QUESTIONS

Q1
Student Performance Analysis — List + Tuple
10 marks

Q2
Customer Name Analysis — String + List
10 marks

Q3
Product Inventory Analysis — List + Tuple
10 marks

Q4
Text Analysis — String + List
10 marks

Q5
Employee Skill Analysis — Tuple + List + String
10 marks

Q6
Transaction Analysis — Tuple + List
10 marks

Q7
Password Security Analysis — String + List
10 marks

Q4 Text Analysis — String + List — 10 marks

A university receives the following feedback:

```
feedback = """
Python programming is easy to learn.
Python programming is powerful.
Learning Python requires practice.
Practice makes programming skills better.
"""
```

Analyze the text.

Tasks

1. Convert the entire text to lowercase.
2. Remove punctuation.
3. Convert the text into a list of words.
4. Count the total number of words.
5. Find the number of unique words.
6. Find the frequency of "python".
7. Find the frequency of "programming".
8. Identify words appearing more than once.
9. Find the longest word.
10. Sort the unique words alphabetically.
11. Find all words containing the letter 'p'.
12. Find the percentage of unique words.

Expected Output:

Total Words: 21

Unique Words: 16

Q8
Code Analysis and Debugging — List + Tuple + String
10 marks

Q9
Word Frequency and Data Analysis — String + List + Tuple
10 marks

Q10
Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

Frequency of python: 3
Frequency of programming: 3

Repeated Words:
python
programming
is
learning
practice

Longest Word:
programming

Words containing 'p':
python
programming
powerful
practice

Your Code

```
feedback = """
Python programming is easy to learn.
Python programming is powerful.
Learning Python requires practice.
Practice makes programming skills better.
"""

# 1. Convert entire text to lowercase
text = feedback.lower()

# 2. Remove punctuation
punctuation = ".,!?;:"

for symbol in punctuation:
    text = text.replace(symbol, "")

# 3. Convert text into a list of words
words = text.split()

print("Words:")
print(words)

# 4. Count total number of words
total_words = len(words)

```

```
print("\nTotal Words:", total_words)

# 5. Find number of unique words
unique_words = set(words)

print("Unique Words:", len(unique_words))

# 6. Frequency of "python"
print("Frequency of python:", words.count

# 7. Frequency of "programming"
print("Frequency of programming:", words.c

# 8. Identify words appearing more than or
print("\nRepeated Words:")

for word in sorted(unique_words):
    if words.count(word) > 1:
        print(word)

# 9. Find the longest word
longest_word = max(unique_words, key=len)

print("\nLongest Word:")
print(longest_word)

# 10. Sort unique words alphabetically
sorted_words = sorted(unique_words)

print("\nUnique Words Alphabetically:")
for word in sorted_words:
    print(word)

# 11. Find words containing the letter 'p'
print("\nWords containing 'p':")

for word in sorted_words:
    if "p" in word:
        print(word)

# 12. Percentage of unique words
percentage = (len(unique_words) / total_wo

print("\nPercentage of Unique Words:",
      format(percentage, ".2f"), "%")
```

Input

stdin input

Output

Run Save

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

Smohit Arman Swain
192572092RC

QUESTIONS

- **Q1** Student Performance Analysis — List + Tuple — 10 marks
- **Q2** Customer Name Analysis — String + List — 10 marks
- **Q3** Product Inventory Analysis — List + Tuple — 10 marks
- **Q4** Text Analysis — String + List — 10 marks
- **Q5** Employee Skill Analysis — Tuple + List + String — 10 marks
- **Q6** Transaction Analysis — Tuple + List — 10 marks
- **Q7** Password Security Analysis — String + List — 10 marks

Q3 Product Inventory Analysis — List + Tuple — 10 marks

A supermarket stores product information as:

products = [
("Laptop", 5, 55000),
("Mouse", 25, 700),
("Keyboard", 8, 1200),
("Monitor", 12, 15000),
("Printer", 4, 12000),
("Headset", 15, 2500)
]

Each tuple contains:

(Product Name, Quantity, Price)

Analyze the inventory.

Tasks

1. Calculate the inventory value of every product.
2. Calculate the total inventory value.
3. Identify products whose quantity is less than 10.
4. Identify products whose price is greater than ₹10,000.
5. Find the most expensive product.
6. Find the product having the highest inventory value.
7. Sort products based on price.
8. Sort products based on quantity.
9. Calculate the total quantity of all products.
10. Identify products contributing more than ₹50,000

Q8
Code Analysis and Debugging — List + Tuple + String
10 marks

Q9
Word Frequency and Data Analysis — String + List + Tuple
10 marks

Q10
Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

to inventory value.

Expected Output:

Inventory Values:

Laptop : 275000

Mouse : 17500

Keyboard : 9600

Monitor : 180000

Printer : 48000

Headset : 37500

Total Inventory Value: 567600

Low Stock:

Laptop

Keyboard

Printer

Price > 10000:

Laptop

Monitor

Printer

Most Expensive:

Laptop

Highest Inventory Value:

Laptop

Total Quantity: 69

Your Code

```
products = [
    ("Laptop", 5, 55000),
    ("Mouse", 25, 700),
    ("Keyboard", 8, 1200),
    ("Monitor", 12, 15000),
    ("Printer", 4, 12000),
    ("Headset", 15, 2500)
]

# 1. Calculate inventory value of every pı
print("Inventory Values:")

inventory_values = {}

```

```
for name, quantity, price in products:
    value = quantity * price
    inventory_values[name] = value
    print(name, ":", value)

# 2. Total inventory value
total_value = sum(inventory_values.values

print("\nTotal Inventory Value:", total_va

# 3. Products whose quantity is less than
print("\nLow Stock:")

for name, quantity, price in products:
    if quantity < 10:
        print(name)

# 4. Products whose price is greater than
print("\nPrice > 10000:")

for name, quantity, price in products:
    if price > 10000:
        print(name)

# 5. Most expensive product
most_expensive = max(products, key=lambda

print("\nMost Expensive:")
print(most_expensive[0])

# 6. Product with highest inventory value
highest_value = max(products, key=lambda p

print("\nHighest Inventory Value:")
print(highest_value[0])

# 7. Sort products based on price
print("\nProducts Sorted by Price:")

sorted_by_price = sorted(products, key=lar

for product in sorted_by_price:
    print(product)

# 8. Sort products based on quantity
print("\nProducts Sorted by Quantity:")

sorted_by_quantity = sorted(products, key=
```

```
63
64 for product in sorted_by_quantity:
65     print(product)
66
67 # 9. Total quantity
68 total_quantity = sum(product[1] for produc
69
70 print("\nTotal Quantity:", total_quantity)
71
72 # 10. Products contributing more than 500(
73 print("\nInventory Value > 50000:")
74
75 for name, quantity, price in products:
76     value = quantity * price
77
78     if value > 50000:
79         print(name)
```

Input

stdin input

Output

Run Save

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

Smohit Arman Swain
192572092RC

CO4_AT2_CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis — List + Tuple
10 marks

Q2
Customer Name Analysis — String + List
10 marks

Q3
Product Inventory Analysis — List + Tuple
10 marks

Q4
Text Analysis — String + List
10 marks

Q5
Employee Skill Analysis — Tuple + List + String
10 marks

Q6
Transaction Analysis — Tuple + List
10 marks

Q7
Password Security Analysis — String + List
10 marks

Q2 Customer Name Analysis — String + List — 10 marks

An e-commerce company receives customer names in inconsistent formats:

names = [
" Arun Kumar ",
"ARUN KUMAR",
" arun kumar",
"Bala Krishnan",
"BALA KRISHNAN ",
"Charan",
" charan ",
"DIVYA DEVI",
"Divya Devi"
]

Analyze the customer data.

Tasks

1. Remove unnecessary leading and trailing spaces.
2. Convert all names to lowercase.
3. Identify duplicate customers.
4. Count the number of unique customers.
5. Find customers whose names contain more than one word.
6. Find the longest customer name.
7. Find customers whose names start with 'a' or 'd'.
8. Count the number of characters in each normalized name.

Q8
Code Analysis and Debugging — List + Tuple + String
10 marks

Q9
Word Frequency and Data Analysis — String + List + Tuple
10 marks

Q10
Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

9. Sort the unique names alphabetically.
10. Determine which customer name occurs most frequently.

Expected Output:

Normalized:
['arun kumar', 'arun kumar', 'arun kumar',
'bala krishnan', 'bala krishnan',
'charan', 'charan',
'divya devi', 'divya devi']

Unique Customers:
['arun kumar', 'bala krishnan', 'charan', 'divya devi']

Number of Unique Customers: 4

Duplicate Customers:
arun kumar
bala krishnan
charan
divya devi

Names with Multiple Words:
arun kumar
bala krishnan
divya devi

Longest Name:
bala krishnan

Starts with A or D:
arun kumar
divya devi

Your Code

```
names = [
    " Arun Kumar ",
    "ARUN KUMAR",
    " arun kumar",
    "Bala Krishnan",
    "BALA KRISHNAN ",
    "Charan",
    " charan ",
    "DIVYA DEVI",
    "Divya Devi"
]
```

```
12
13 # 1 & 2. Remove spaces and convert to lowe
14 normalized = []
15
16 for name in names:
17     name = name.strip().lower()
18     normalized.append(name)
19
20 print("Normalized:")
21 print(normalized)
22
23 # 3. Identify duplicate customers
24 duplicates = []
25
26 for name in normalized:
27     if normalized.count(name) > 1 and name
28         duplicates.append(name)
29
30 # 4. Count unique customers
31 unique_customers = []
32
33 for name in normalized:
34     if name not in unique_customers:
35         unique_customers.append(name)
36
37 print("\nUnique Customers:")
38 print(unique_customers)
39
40 print("Number of Unique Customers:", len(u
41
42 print("\nDuplicate Customers:")
43 for name in duplicates:
44     print(name)
45
46 # 5. Names containing more than one word
47 print("\nNames with Multiple Words:")
48
49 for name in unique_customers:
50     if len(name.split()) > 1:
51         print(name)
52
53 # 6. Find the longest customer name
54 longest_name = max(unique_customers, key=l
55
56 print("\nLongest Name:")
57 print(longest_name)
58
59 # 7. Names starting with 'a' or 'd'
```

```
print("\nStarts with A or D:")

for name in unique_customers:
    if name.startswith("a") or name.starts
        print(name)

# 8. Number of characters in each normali
print("\nCharacter Count:")

for name in unique_customers:
    print(name, ":", len(name))

# 9. Sort unique names alphabetically
sorted_names = sorted(unique_customers)

print("\nAlphabetically Sorted:")
for name in sorted_names:
    print(name)

# 10. Customer name occurring most frequer
most_frequent = max(unique_customers, key=

print("\nMost Frequent Customer:")
print(most_frequent)
```

Input

stdin input

Output

Run Save

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

Smohit Arman Swain
192572092RC

CO4_AT2_CASE STUDY

← Back

QUESTIONS

Q1 ✓
Student Performance Analysis — List + Tuple
10 marks

Q2 ✓
Customer Name Analysis — String + List
10 marks

Q3 ✓
Product Inventory Analysis — List + Tuple
10 marks

Q4 ✓
Text Analysis — String + List
10 marks

Q5 ✓
Employee Skill Analysis — Tuple + List + String
10 marks

Q6 ✓
Transaction Analysis — Tuple + List
10 marks

Q7 ✓
Password Security Analysis — String + List
10 marks

Q1 Student Performance Analysis — List + Tuple — 10 marks

A college stores the following student records:

```
students = [
("Arun", [78, 85, 92, 67]),
("Bala", [45, 67, 55, 49]),
("Charan", [90, 92, 88, 95]),
("Divya", [65, 72, 70, 68]),
("Esha", [88, 76, 91, 84])
]
```

Analyze the student data and perform the following operations:

1. Calculate the total and average marks of every student.
2. Identify the student with the highest total.
3. Identify the student with the lowest average.
4. Find students whose average is greater than the class average.
5. Find students who scored above 80 in at least three subjects.
6. Find the highest mark in each subject.
7. Identify students who scored below 50 in any subject.
8. Sort the students based on their average marks in descending order.
9. Identify the top two students.
10. Explain which parts of the structure are mutable

Q8
Code Analysis and Debugging — List + Tuple + String
10 marks

Q9
Word Frequency and Data Analysis — String + List + Tuple
10 marks

Q10
Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

and which are immutable.

Expected Output:

Student Totals:

Arun : 322

Bala : 216

Charan : 365

Divya : 275

Esha : 339

Student Averages:

Arun : 80.50

Bala : 54.00

Charan : 91.25

Divya : 68.75

Esha : 84.75

Highest Total: Charan

Lowest Average: Bala

Above Class Average:

Arun, Charan, Esha

Students with 3 or more marks above 80:

Charan

Subject-wise Highest:

Subject 1: 90

Subject 2: 92

Subject 3: 92

Subject 4: 95

Students with a mark below 50:

Bala

Top 2:

Charan

Esha

Your Code

```
students = [
    ("Arun", [78, 85, 92, 67]),
    ("Bala", [45, 67, 55, 49]),
    ("Charan", [90, 92, 88, 95]),
    ("Divya", [65, 72, 70, 68]),
```

```
    ("Esha", [88, 76, 91, 84])
]

# 1. Calculate total and average
totals = {}
averages = {}

for name, marks in students:
    total = sum(marks)
    average = total / len(marks)
    totals[name] = total
    averages[name] = average

print("Student Totals:")
for name in totals:
    print(name, ":", totals[name])

print("\nStudent Averages:")
for name in averages:
    print(name, ":", format(averages[name

# 2. Student with highest total
highest_total_student = max(totals, key=t
print("\nHighest Total:", highest_total_s

# 3. Student with lowest average
lowest_average_student = min(averages, ke
print("Lowest Average:", lowest_average_s

# 4. Students above class average
class_average = sum(averages.values()) /

above_class_average = []

for name in averages:
    if averages[name] > class_average:
        above_class_average.append(name)

print("\nAbove Class Average:")
print(", ".join(above_class_average))

# 5. Students with 3 or more marks above
above_80 = []

for name, marks in students:
    count = 0
    for mark in marks:
        if mark > 80:
```

```
            count += 1

    if count >= 3:
        above_80.append(name)

print("\nStudents with 3 or more marks ab
print(", ".join(above_80))

# 6. Highest mark in each subject
print("\nSubject-wise Highest:")

for i in range(4):
    highest = max(marks[i] for name, mark
    print("Subject", i + 1, ":", highest)

# 7. Students with a mark below 50
below_50 = []

for name, marks in students:
    for mark in marks:
        if mark < 50:
            below_50.append(name)
            break

print("\nStudents with a mark below 50:")
print(", ".join(below_50))

# 8. Sort students by average in descendi
sorted_students = sorted(
    students,
    key=lambda student: sum(student[1]) /
    reverse=True
)

print("\nStudents sorted by average (desc

for name, marks in sorted_students:
    average = sum(marks) / len(marks)
    print(name, ":", format(average, ".2f

# 9. Top two students
print("\nTop 2:")

for name, marks in sorted_students[:2]:
    print(name)

# 10. Mutable and immutable parts
print("\n10. Mutable and Immutable:")
```

```
print("List 'students' is mutable.")
print("The inner marks lists are mutable.
print("Each student record is a tuple, wh
print("Student names and marks are intege
```

Input

stdin input

Output

Run Save